

Trabajo Práctico 5

Capa de transporte - Parte 1

Inciso 1

¿Cuál es la función de la capa de transporte?

Respuesta:

La función principal de la **capa de transporte** es proporcionar una **comunicación lógica entre procesos de aplicación** que se ejecutan en hosts diferentes

Inciso 2

Describe la estructura del segmento TCP y UDP.

Respuesta:

1. Estructura del segmento UDP

El segmento **UDP (User Datagram Protocol)** tiene una cabecera minimalista y liviana de solo **8 bytes (64 bits)** de longitud fija, dividida en 4 campos de 16 bits cada uno:

- **Número de puerto de origen (16 bits / 2 bytes):** Identifica el puerto del proceso emisor y sirve como dirección de retorno para el receptor. En IPv4 este campo es opcional y puede establecerse en cero si no se requiere respuesta.
- **Número de puerto de destino (16 bits / 2 bytes):** Identifica el puerto del proceso receptor en el host de destino .
- **Longitud (16 bits / 2 bytes):** Especifica el tamaño total del segmento UDP en bytes, sumando tanto la cabecera (8 bytes) como los datos de la aplicación.
- **Suma de comprobación / Checksum (16 bits / 2 bytes):** Utilizado para detectar si se han introducido errores en los bits del segmento durante la transmisión. Cubre la cabecera UDP, la carga útil de datos y un pseudo-

encabezado IP. En IPv4 es opcional (se puede desactivar colocando todo en ceros), pero en IPv6 es obligatorio.

- **Datos de la aplicación (carga útil / payload):** Contiene el mensaje propiamente dicho enviado por la capa de aplicación (como consultas DNS o bloques de datos multimedia).
-

2. Estructura del segmento TCP

El segmento **TCP (Transmission Control Protocol)** consta de una cabecera estándar de **20 bytes (160 bits)** de longitud mínima base, la cual puede extenderse mediante el campo opcional hasta los 60 bytes:

- **Número de puerto de origen (16 bits / 2 bytes):** Identifica el proceso emisor que genera el flujo de datos.
- **Número de puerto de destino (16 bits / 2 bytes):** Identifica el proceso receptor al que va dirigido el segmento.
- **Número de secuencia (32 bits / 4 bytes):** Indica el número del primer byte de datos contenido en este segmento dentro del flujo continuo de bytes de la sesión.
- **Número de reconocimiento / ACK Number (32 bits / 4 bytes):**
Transporta el número de secuencia del siguiente byte que el host emisor del ACK espera recibir de la otra parte, implementando un mecanismo de reconocimientos acumulativos.
- **Longitud de cabecera / Data Offset (4 bits):** Especifica la longitud total de la cabecera TCP medida en palabras de 32 bits (4 bytes), lo que permite determinar dónde comienzan los datos de la aplicación.
- **Reservado (3 a 6 bits):** Bits reservados para uso futuro.
- **Bits de control / Flags (6 bits principales):**
 - **URG (Urgent):** Indica que el campo Puntero de datos urgentes contiene información válida.
 - **ACK (Acknowledgement):** Indica que el valor del campo Número de reconocimiento es válido.
 - **PSH (Push):** Solicita al receptor que pase los datos acumulados a la capa de aplicación de forma inmediata sin esperar a llenar los buffers.

- **RST (Reset)**: Resetea o aborta la conexión TCP ante errores o situaciones anómalas.
 - **SYN (Synchronize)**: Utilizado para sincronizar los números de secuencia iniciales durante el establecimiento de la conexión (*three-way handshake*).
 - **FIN (Finish)**: Utilizado para iniciar el cierre ordenado de la conexión.
 - **Ventana de recepción / Window Size (16 bits / 2 bytes)**: Utilizado para el **control de flujo**; indica la cantidad de bytes que el receptor está dispuesto a aceptar en su buffer de recepción sin saturarse.
 - **Suma de comprobación / Checksum (16 bits / 2 bytes)**: Proporciona detección de errores en la cabecera, en los datos y en el pseudo-encabezado IP.
 - **Puntero de datos urgentes / Urgent Pointer (16 bits / 2 bytes)**: Apunta al último byte de los datos urgentes dentro de la carga útil cuando el flag **URG** está activo.
 - **Opciones (longitud variable, de 0 a 40 bytes)**: Permite negociar parámetros como el **MSS** (*Maximum Segment Size*), el factor de escala de ventana (*Window Scale*), reconocimientos selectivos (**SACK**) y marcas temporales (*Timestamps*).
 - **Datos de la aplicación (carga útil / payload)**: Fragmento del flujo de bytes de la aplicación delimitado por el tamaño MSS.
-

Inciso 3

¿Cuál es el objetivo del uso de puertos en el modelo TCP/IP?

El objetivo principal del uso de puertos en el modelo TCP/IP es extender la comunicación de host a host hacia una comunicación de proceso a proceso.

Esto permite dos funciones:

- **Multiplexar y desmultiplexar**: Reunir los datos de distintas aplicaciones en el host origen para enviarlos a la red y, en el destino, entregar cada segmento al proceso o *socket* de aplicación que le corresponde.

- **Ejecutar múltiples aplicaciones en simultáneo:** Permitir que un solo equipo (con una única IP) mantenga varias conexiones de red activas al mismo tiempo (como el navegador web, el correo y DNS), asignando un número de puerto distinto a cada servicio.

Inciso 4

Compare TCP y UDP en cuanto a:

1. Confiabilidad
2. Multiplexación.
3. Orientado a la conexión.
4. Controles de congestión.
5. Utilización de puertos.

Confiabilidad

- **TCP:** Proporciona un servicio de transferencia de datos fiable (rdt). Garantiza que el flujo de bytes entregado por el emisor llegue a la aplicación de destino en orden, sin errores, sin duplicados y sin pérdidas. Para lograr esto, utiliza números de secuencia, reconocimientos acumulativos (ACKs), temporizadores de retransmisión y mecanismos de control de flujo que evitan saturar el buffer del receptor.
 - **UDP:** Es un servicio básico de "mejor esfuerzo" (best effort) no fiable. No garantiza que los datagramas lleguen a su destino, ni que lo hagan en el orden enviado, ni previene la duplicación o pérdida. Solo cuenta con una suma de comprobación (checksum) para la detección básica de errores de bits. Si una aplicación basada en UDP requiere fiabilidad, debe implementar dichos controles directamente en la capa de aplicación.
-

Multiplexación y Desmultiplexación

- **TCP:** Realiza la desmultiplexación utilizando una **4-upla (tupla de 4 elementos)**: (IP origen, puerto origen, IP destino, puerto destino) . Esto permite que dos segmentos TCP procedentes de diferentes direcciones IP o puertos de origen se dirijan a sockets de conexión distintos en el servidor, permitiendo gestionar múltiples

conexiones concurrentes sobre un mismo puerto (por ejemplo, el puerto 80).

- **UDP:** Realiza la desmultiplexación utilizando una **2-tupla (tupla de 2 elementos)**: (IP destino, puerto destino) . Todos los datagramas UDP dirigidos a la misma IP y puerto de destino se entregarán al mismo socket en el host receptor, independientemente de la IP o del puerto de origen desde el que provengan.
-

Orientado a la conexión

- **TCP:** Es un protocolo orientado a la conexión. Antes de transferir datos de la capa de aplicación, el cliente y el servidor ejecutan una fase de negociación o saludo de tres vías (three-way handshake) para establecer la conexión e inicializar parámetros de estado (números de secuencia iniciales, buffers y ventanas) en ambos extremos. Al finalizar la transmisión, la conexión se cierra mediante un intercambio formal de mensajes.
 - **UDP:** Es un protocolo sin conexión. Inicia la transmisión directamente sin formalidades preliminares ni negociación previa. No mantiene información sobre el estado de la conexión en el emisor ni en el receptor, tratando cada segmento de forma totalmente independiente.
-

Controles de congestión

- **TCP:** Incluye un mecanismo activo de **control de congestión** de extremo a extremo. Supervisa el estado de la red mediante una *ventana de congestión* (`VentCongestion`) y limita la velocidad de transmisión del emisor. Emplea algoritmos como AIMD (incremento aditivo / decrecimiento multiplicativo) para reducir la velocidad al detectar pérdidas o expiraciones de temporizadores, buscando además distribuir el ancho de banda de forma equitativa entre las conexiones que comparten un enlace.
- **UDP:** No incluye ningún mecanismo de control de congestión. La aplicación emite datos hacia la capa de red a la velocidad que considere

oportuna, sin regular su tasa de envío incluso ante situaciones de congestión severa o pérdidas elevadas de paquetes en la red.

Utilización de puertos

- **Ambos protocolos (TCP y UDP):** Emplean campos de 16 bits en sus cabeceras para indicar el *número de puerto de origen* y el *número de puerto de destino* (con valores que van de 0 a 65535, reservando los puertos de 0 a 1023 para servicios bien conocidos).
 - **Diferencias operativas en el uso de puertos:**
 - En **TCP**, el puerto del servidor suele estar asociado a un *socket de acogida* en escucha (`LISTEN`), y por cada nueva conexión aceptada se crea un nuevo *socket de conexión* específico identificado por la 4-upla.
 - En **UDP**, el servidor generalmente vincula un único socket a su número de puerto mediante `bind()` , utilizando ese mismo puerto y socket para recibir los datagramas de cualquier cliente y extraer el puerto y la IP de origen como dirección de retorno
-

Inciso 5

La PDU de la capa de transporte es el segmento. Sin embargo, en algunos contextos suele utilizarse el término datagrama. Indique cuando

Respuesta

Suele utilizarse en la capa de transporte en los siguientes casos concretos:

1. Al referirse al protocolo UDP

Aunque de forma genérica se usa **segmento** para cualquier PDU (Protocolo de unidad de trabajo) de la capa de transporte, en la especificación RFC 768 y en la documentación técnica la unidad de datos de UDP se denomina formalmente datagrama UDP. Esto se debe a su naturaleza sin conexión y no confiable.

2. En la API de Sockets de programación

En la programación de red, se distingue explícitamente entre:

- **Sockets de flujo (Stream Sockets - SOCK_STREAM)**: orientados a conexión y basados en el flujo continuo de bytes de TCP (segmentos).
- **Sockets de datagrama (Datagram Sockets - SOCK_DGRAM)**: orientados a mensajes independientes sin conexión para UDP (datagramas)

3. En otros protocolos de transporte sin conexión

Existen otros protocolos de transporte de Internet que adoptan el término datagrama en su propia definición, como **DCCP** (*Datagram Congestion Control Protocol*), diseñado para aplicaciones de streaming que requieren control de congestión pero sobre un servicio de datagramas no fiable.

Inciso 6

Describe el saludo de tres vías de TCP. ¿UDP tiene esta característica?

Respuesta

Es el mecanismo que utiliza el protocolo **TCP** para establecer una conexión orientada a conexión antes de comenzar la transferencia de datos entre el cliente y el servidor.

Su objetivo principal es inicializar las variables de estado en ambos extremos, asignar buffers y sincronizar los **números de secuencia iniciales** de cada host.

El procedimiento consta de tres intercambios de mensajes¹³:

1. Paso 1: Cliente → Servidor (**SYN**)

- El cliente envía un segmento TCP especial de control con la bandera **SYN = 1**.
- Selecciona de forma aleatoria un número de secuencia inicial (`cliente_nsi`) y no transporta datos de la capa de aplicación⁴⁵.

2. Paso 2: Servidor → Cliente (**SYN-ACK**)

- Al recibir el `SYN`, el servidor asigna los buffers y variables para la conexión.

- Responde con un segmento que tiene las banderas **SYN = 1** y **ACK = 1**.
- Confirma la recepción del cliente colocando en el campo de reconocimiento `ack = cliente_nsi + 1` y especifica su propio número de secuencia inicial (`servidor_nsi`).

3. Paso 3: Cliente → Servidor (**ACK**)

- Al recibir el SYN-ACK, el cliente también asigna sus buffers y variables de estado.
- Envía un segmento con **SYN = 0** y **ACK = 1**, confirmando el número del servidor con `ack = servidor_nsi + 1`.
- Este tercer segmento **ya puede transportar datos** de la capa de aplicación.

UDP no posee un saludo de tres vías ni ningún procedimiento de establecimiento de conexión.

- **UDP es un protocolo sin conexión (connectionless)**: envía los datagramas directamente hacia el socket de destino sin acordar parámetros previos ni verificar si el receptor está listo.
- **No mantiene estado**: no crea ni administra buffers de conexión, números de secuencia ni variables de control en el emisor o receptor.
- Cada datagrama enviado por UDP se trata de manera totalmente independiente de los demás.

Inciso 7

Investigue qué es el ISN (Initial Sequence Number). Relaciónelo con el saludo de tres vías.

Respuesta

El **ISN** (Initial Sequence Number o Número de Secuencia Inicial) es el valor numérico asignado de forma aleatoria por cada host (cliente y servidor) para marcar el número de secuencia con el que comenzará su flujo de datos en una conexión TCP.

A diferencia de lo que se podría suponer, TCP no comienza sus números de

secuencia en 0, sino que cada extremo genera un ISN único al momento de negociar la conexión.

¿Por qué el ISN se elige de forma aleatoria?

- Evitar la confusión con segmentos de conexiones previas: Si dos hosts cierran una conexión TCP y abren inmediatamente otra nueva utilizando la misma tupla de IP y puertos, un paquete retrasado que pertenecía a la sesión anterior podría llegar a la nueva sesión. Al usar un ISN aleatorio en cada nueva conexión, los números de secuencia difieren lo suficiente como para que el receptor identifique y descarte cualquier segmento antiguo remanente en la red.
- **Seguridad y prevención de ataques:** Si el ISN fuera predecible (por ejemplo, si siempre empezara en 0 o se incrementara en un valor fijo), un atacante podría realizar ataques de suplantación de IP (*IP Spoofing*) o inyección de código.

Relación del ISN con el Saludo de Tres Vías (Three-Way Handshake)

La sincronización de los números de secuencia iniciales es, precisamente, uno de los objetivos principales del saludo de tres vías:

1. Paso 1 (SYN del cliente):

- El cliente selecciona aleatoriamente su número de secuencia inicial (`cliente_nsi`) y lo coloca en el campo *Número de secuencia* del segmento SYN .

2. Paso 2 (SYN-ACK del servidor):

- El servidor recibe el SYN , reserva recursos para la conexión y elige su propio número de secuencia inicial (`servidor_nsi`) para el flujo en sentido inverso.
- Responde con un segmento SYN-ACK colocando en el campo *Número de reconocimiento (ACK)* el valor `cliente_nsi + 1` , confirmando que ha recibido el SYN del cliente y sincronizando el ISN de este.

3. Paso 3 (ACK del cliente):

- El cliente recibe el SYN-ACK y responde con un segmento ACK introduciendo en el campo de reconocimiento el valor `servidor_nsi + 1` .

- Con esto, el cliente confirma la recepción del ISN del servidor.
-

Inciso 8

Investigue qué es el MSS. ¿Cuándo y cómo se negocia?

Respuesta:

El MSS (Maximum Segment Size o Tamaño Máximo de Segmento) es la cantidad máxima de datos útiles de la capa de aplicación (**payload**) que un host puede enviar o recibir en un único segmento TCP.

Cálculo:

- Se deriva de la MTU (Maximum Transmission Unit) de la capa de enlace local o de la ruta (Path MTU) para garantizar que el segmento completo no sufra fragmentación en la capa de red (IP):

$$SS = T - \text{abecera IP (20 btes)} - \text{abecera TP (20 btes)}$$

- **Ejemplo típico:** En redes Ethernet, donde la MTU habitual es de **1500 bytes**, la resta de los 40 bytes de encabezados estándar deja un valor común de **MSS de 1460 bytes**.

¿Cuándo y cómo se negocia?

El MSS se informa **durante la fase de establecimiento de la conexión**, concretamente dentro del **saludo de tres vías (three-way handshake)**.

¿Cómo?

1. **Inclusión en el campo Opciones:** Tanto el cliente como el servidor utilizan el campo de **Opciones** de la cabecera TCP (tipo de opción 2) para especificar su valor de MSS.
2. **Intercambio en los segmentos de control:**
 - El cliente envía su MSS deseado en el segmento inicial **SYN**.
 - El servidor responde con su propio valor de MSS en el segmento **SYN-ACK**
3. **Mecanismo de anuncio (sin negociación bidireccional estricta):**

- Más que una negociación interactiva, **cada extremo anuncia el tamaño máximo de datos que sus buffers y su capa de enlace pueden recibir.**
- Al finalizar el intercambio, el emisor limita el tamaño de los datos de cada segmento para que no supere el valor de MSS anunciado por el receptor del extremo opuesto.

Inciso 9

Utilice el comando ss (reemplazo de netstat) para obtener la siguiente información de su PC:

1. Para listar las comunicaciones TCP establecidas.
2. Para listar las comunicaciones UDP establecidas.
3. Obtener sólo los servicios TCP que están esperando comunicaciones
4. Obtener sólo los servicios UDP que están esperando comunicaciones.
5. Repetir los anteriores para visualizar el proceso del sistema asociado a la conexión.
6. Obtenga la misma información planteada en los items anteriores usando el comando netstat.

Respuesta

Inciso 9.1

```
ss -t state established
```

Inciso 9.2

```
ss -u state established
```

Inciso 9.3

```
ss -tl
```

- `-t` : protocolo TCP.
- `-l` : sockets en estado de escucha (`LISTEN`).

Inciso 9.4

```
ss -ul
```

- -u: protocolo UDP.
- -l: sockets a la espera de datagramas entrantes.

Inciso 9.5

Para ver el PID y nombre del ejecutable se agrega `-p` (requiere privilegios de administrador `sudo` para ver procesos de otros usuarios):

- **TCP establecidas:** `sudo ss -tp state established`
- **UDP establecidas:** `sudo ss -up state established`
- **TCP a la escucha:** `sudo ss -tlp`
- **UDP a la espera:** `sudo ss -ulp`

Inciso 9.6

- **Comunicaciones TCP establecidas:**

```
netstat -tn | grep ESTABLISHED
```

O bien `netstat -t` para verlas con nombres de servicios en lugar de números).

- **Comunicaciones UDP establecidas:**

```
netstat -un | grep ESTABLISHED
```

(Al igual que en `ss`, muestra sockets UDP vinculados mediante `connect`).

- **Servicios TCP esperando comunicaciones (LISTEN):**
 - `-t`: protocolo TCP.
 - `-l`: conexiones a la escucha (`LISTEN`).
 - `-n`: muestra direcciones y puertos numéricos.

```
netstat -tln
```

- **Servicios UDP esperando comunicaciones:**
 - `-u` : protocolo UDP.
 - `-l` : sockets a la escucha (en UDP no figura formalmente la palabra LISTEN en el estado, pero `-l` los aísla).

```
netstat -uln
```

- **Visualizar el proceso del sistema asociado:** Al igual que con `ss`, se incorpora el modificador `-p` ejecutando con privilegios de `root` (`sudo`):
 - **TCP establecidas:** `sudo netstat -tnp | grep ESTABLISHED`
 - **UDP establecidas:** `sudo netstat -unp | grep ESTABLISHED`
 - **TCP a la escucha:** `sudo netstat -tlnp`
 - **UDP a la espera:** `sudo netstat -ulnp`
-

Inciso 10

[Documentación hping3](#)

¿Qué sucede si llega un segmento TCP con el flag SYN activo a un host que no tiene ningún proceso esperando en el puerto destino de dicho segmento (es decir, el puerto destino no está en estado LISTEN)?

1. Utilice hping3 para enviar paquetes TCP al puerto destino 22 de la máquina virtual con el flag SYN activado.
2. Utilice hping3 para enviar paquetes TCP al puerto destino 40 de la máquina virtual con el flag SYN activado.
3. ¿Qué diferencias nota en las respuestas obtenidas en los dos casos anteriores? ¿Puede explicar a qué se debe? (Ayuda: utilice el comando `ss` visto anteriormente).

Respuesta

Si un segmento TCP con el flag **SYN** activo llega a un puerto de destino donde no hay ningún proceso escuchando (es decir, el puerto no está en estado LISTEN), el host receptor responde enviando un segmento especial de reinicio con el flag **RST**(Reset) activado.

Con este mensaje, el host le indica formalmente al emisor que no existe ningún socket ni aplicación disponible para esa solicitud en ese puerto, pidiéndole que no reenvíe el segmento.

Inciso 10.1

```
sudo hping3 -S -p 22 -c 3 <IP_DESTINO>
```

- `-S` : Activa el bit o bandera **SYN** dentro del campo de flags de la cabecera TCP. Sirve para simular el primer paso del saludo de tres vías (*three-way handshake*) con el que se solicita iniciar una conexión TCP.
- `-p 22` (o `--destport`): Especifica el **puerto de destino** en el host remoto. En este caso, el puerto 22 corresponde por convención al servicio SSH (*Secure Shell*)
- `-c 3` (o `--count`): Define el **conteo o cantidad total de paquetes** a transmitir (`packet count`). Una vez enviados los 3 paquetes, el programa detiene la transmisión automáticamente en lugar de continuar indefinidamente.

```
redes@debian:~$ sudo hping3 -S -p 22 -c 3 127.0.0.1
HPING 127.0.0.1 (lo 127.0.0.1): S set, 40 headers + 0 data bytes
len=44 ip=127.0.0.1 ttl=64 DF id=0 sport=22 flags=SA seq=0 win=65495 rtt=7.9 ms
len=44 ip=127.0.0.1 ttl=64 DF id=0 sport=22 flags=SA seq=1 win=65495 rtt=2.8 ms
len=44 ip=127.0.0.1 ttl=64 DF id=0 sport=22 flags=SA seq=2 win=65495 rtt=4.7 ms

--- 127.0.0.1 hping statistic ---
3 packets transmitted, 3 packets received, 0% packet loss
round-trip min/avg/max = 2.8/5.2/7.9 ms
redes@debian:~$
```

Inciso 10.2

```
sudo hping3 -S -p 40 -c 3 <IP_DESTINO>
```

```
redes@debian:~$ sudo hping3 -S -p 40 -c 3 127.0.0.1
HPING 127.0.0.1 (lo 127.0.0.1): S set, 40 headers + 0 data bytes
len=40 ip=127.0.0.1 ttl=64 DF id=0 sport=40 flags=RA seq=0 win=0 rtt=7.9 ms
len=40 ip=127.0.0.1 ttl=64 DF id=0 sport=40 flags=RA seq=1 win=0 rtt=3.8 ms
len=40 ip=127.0.0.1 ttl=64 DF id=0 sport=40 flags=RA seq=2 win=0 rtt=2.4 ms

--- 127.0.0.1 hping statistic ---
3 packets transmitted, 3 packets received, 0% packet loss
round-trip min/avg/max = 2.4/4.7/7.9 ms
redes@debian:~$
```

Inciso 10.3

```

redes@debian:~$ sudo hping3 -S -p 22 -c 3 127.0.0.1
HPING 127.0.0.1 (lo 127.0.0.1): S set, 40 headers + 0 data bytes
len=44 ip=127.0.0.1 ttl=64 DF id=0 sport=22 flags=SA seq=0 win=65495 rtt=7.9 ms
len=44 ip=127.0.0.1 ttl=64 DF id=0 sport=22 flags=SA seq=1 win=65495 rtt=2.8 ms
len=44 ip=127.0.0.1 ttl=64 DF id=0 sport=22 flags=SA seq=2 win=65495 rtt=4.7 ms

--- 127.0.0.1 hping statistic ---
3 packets transmitted, 3 packets received, 0% packet loss
round-trip min/avg/max = 2.8/5.2/7.9 ms
redes@debian:~$ sudo hping3 -S -p 40 -c 3 127.0.0.1
HPING 127.0.0.1 (lo 127.0.0.1): S set, 40 headers + 0 data bytes
len=40 ip=127.0.0.1 ttl=64 DF id=0 sport=40 flags=RA seq=0 win=0 rtt=7.9 ms
len=40 ip=127.0.0.1 ttl=64 DF id=0 sport=40 flags=RA seq=1 win=0 rtt=3.8 ms
len=40 ip=127.0.0.1 ttl=64 DF id=0 sport=40 flags=RA seq=2 win=0 rtt=2.4 ms

--- 127.0.0.1 hping statistic ---
3 packets transmitted, 3 packets received, 0% packet loss
round-trip min/avg/max = 2.4/4.7/7.9 ms
redes@debian:~$

```

Comparando las salidas de `hping3` hacia el puerto 22 y hacia el puerto 40:

- **Flags TCP:**
 - **Puerto 22:** Responde con `flags=SA` (**SYN + ACK**). Indica que el host remoto acepta la solicitud y responde con el segundo paso del three-way handshake.
 - **Puerto 40:** Responde con `flags=RA` (**RST + ACK**). Indica que el host rechaza la solicitud de conexión mediante un reinicio/reset.
- **Tamaño del mensaje (`len`):**
 - **Puerto 22:** La respuesta tiene `len=44` bytes (20 bytes de cabecera IP + 24 bytes de cabecera TCP). Estos 4 bytes adicionales en TCP se deben a que en la negociación activa del inicio de conexión se incluyen opciones TCP (por ejemplo, MSS).
 - **Puerto 40:** La respuesta tiene `len=40` bytes (20 bytes de cabecera IP + 20 bytes de cabecera TCP base). Un paquete RST no negocia opciones adicionales, por lo que la cabecera TCP se mantiene en su tamaño mínimo estándar de 20 bytes.
- **Tamaño de ventana (`win`):**
 - **Puerto 22:** Informa un tamaño de ventana de recepción `win=65495` (buffer disponible para recibir datos).
 - **Puerto 40:** Informa `win=0`, ya que al abortar/rechazar la conexión no hay ningún buffer de recepción asignado.
- **Puerto de origen de la respuesta (`sport`):**
 - En el primer caso es `sport=22` y en el segundo `sport=40`

Salida del ss:

```

redes@debian:~$ ss -tlpn
State      Recv-Q    Send-Q      Local Address:Port      Peer Address:Port      Process
LISTEN     0         5            127.0.0.1:4038          0.0.0.0:*
LISTEN     0        128           0.0.0.0:22             0.0.0.0:*
LISTEN     0        128        127.0.0.1:631          0.0.0.0:*
LISTEN     0       4096           [::1]:50051           [::]:*
LISTEN     0       4096  [::ffff:127.0.0.1]:50051  *:50051
LISTEN     0        128           [::]:22               [::]:*
LISTEN     0        128           [::1]:631             [::]:*
redes@debian:~$

```

- `-t` : Filtra sockets del protocolo **TCP**.
- `-l` : Muestra únicamente los sockets en estado de escucha (`LISTEN`).
- `-p` : Muestra el proceso asociado (requiere privilegios o `sudo`).
- `-n` : Muestra direcciones y puertos de forma numérica (evita traducir el puerto 22 a la palabra `ssh`).

Inciso 11

¿Qué sucede si llega un datagrama UDP a un host que no tiene ningún proceso

esperando en el puerto destino de dicho datagrama (es decir, que dicho puerto no está en estado `LISTEN`)?

1. Utilice `hping3` para enviar datagramas UDP al puerto destino 5353 de la máquina virtual.
2. Utilice `hping3` para enviar datagramas UDP al puerto destino 40 de la máquina virtual.
3. ¿Qué diferencias nota en las respuestas obtenidas en los dos casos anteriores? ¿Puede explicar a qué se debe? (Ayuda: utilice el comando `ss` visto anteriormente)

Respuesta

Cuando un datagrama UDP llega a un host de destino donde no existe ningún proceso escuchando en el puerto indicado (sin socket UDP asociado), ocurren los siguientes eventos:

1. Generación de un mensaje de error ICMP
 - Dado que UDP es un protocolo sin conexión y carece de banderas de control propias en su cabecera (a diferencia de TCP, que enviaría un segmento `RST`), la pila de protocolos de red del host receptor recurre

a la capa de red y genera un mensaje **ICMP (Internet Control Message Protocol)**.

- Específicamente, el receptor devuelve al host de origen un paquete:
 - **ICMP Destination Unreachable (Tipo 3, Código 3: Port Unreachable / Puerto inalcanzable)**.

2. Contenido del mensaje ICMP

1. El mensaje de error ICMP devuelto incluye la **cabecera IP original y los primeros 8 bytes de la cabecera del datagrama UDP** que provocó el fallo. Esto permite que el sistema operativo emisor identifique con precisión qué proceso y socket local generaron el datagrama rechazado.

```
sudo hping3 -2 -p puerto -c 3 127.0.0.1
```

-2 = UDP

Inciso 11.1

```
redes@debian:~$ sudo hping3 -2 -p 5353 -c 3 127.0.0.1
HPING 127.0.0.1 (lo 127.0.0.1): udp mode set, 28 headers + 0 data bytes

--- 127.0.0.1 hping statistic ---
3 packets transmitted, 0 packets received, 100% packet loss
round-trip min/avg/max = 0.0/0.0/0.0 ms
redes@debian:~$
```

Inciso 11.2

```
redes@debian:~$ sudo hping3 -2 -p 40 -c 3 127.0.0.1
HPING 127.0.0.1 (lo 127.0.0.1): udp mode set, 28 headers + 0 data bytes
ICMP Port Unreachable from ip=127.0.0.1 name=localhost
status=0 port=2727 seq=0
ICMP Port Unreachable from ip=127.0.0.1 name=localhost
status=0 port=2728 seq=1
ICMP Port Unreachable from ip=127.0.0.1 name=localhost
status=0 port=2729 seq=2

--- 127.0.0.1 hping statistic ---
3 packets transmitted, 3 packets received, 0% packet loss
round-trip min/avg/max = 5.1/8.4/11.9 ms
redes@debian:~$
```

Inciso 11.3

```

redes@debian:~$ sudo hping3 -2 -p 40 -c 3 127.0.0.1
HPING 127.0.0.1 (lo 127.0.0.1): udp mode set, 28 headers + 0 data bytes
ICMP Port Unreachable from ip=127.0.0.1 name=localhost
status=0 port=2727 seq=0
ICMP Port Unreachable from ip=127.0.0.1 name=localhost
status=0 port=2728 seq=1
ICMP Port Unreachable from ip=127.0.0.1 name=localhost
status=0 port=2729 seq=2

--- 127.0.0.1 hping statistic ---
3 packets transmitted, 3 packets received, 0% packet loss
round-trip min/avg/max = 5.1/8.4/11.9 ms
redes@debian:~$ ss -ulpn
State      Recv-Q      Send-Q       Local Address:Port      Peer Address:Port      Process
UNCONN     0            0            127.0.0.1:4038          0.0.0.0:*
UNCONN     0            0            0.0.0.0:631            0.0.0.0:*
UNCONN     0            0            0.0.0.0:37859          0.0.0.0:*
UNCONN     0            0            0.0.0.0:5353           0.0.0.0:*
UNCONN     0            0            [::]:47922             [::]:*
UNCONN     0            0            [::]:5353              [::]:*
redes@debian:~$ sudo hping3 -2 -p 5353 -c 3 127.0.0.1
HPING 127.0.0.1 (lo 127.0.0.1): udp mode set, 28 headers + 0 data bytes

--- 127.0.0.1 hping statistic ---
3 packets transmitted, 0 packets received, 100% packet loss
round-trip min/avg/max = 0.0/0.0/0.0 ms
redes@debian:~$

```

- Para los datagramas al puerto 5353, se observa 100% de pérdida de paquetes, sin recibir ninguna respuesta. Se debe a que, a diferencia de TCP, el protocolo UDP no está orientado a la conexión y no realiza un saludo de tres vías (*three-way handshake*) ni devuelve confirmaciones (ACKs) de forma automática.
- Para los datagramas al puerto al cuerpo 40 se obtiene una respuesta inmediata reflejada como 0% packet loss, detallando: ICMP Port Unreachable from ip=127.0.0.1.

El paquete llegó a la máquina de destino, pero el sistema operativo no encontró ningún proceso con un socket UDP activo vinculado al puerto 40. Como la capa de transporte en UDP no dispone de señalización de control propia (como el flag RST de TCP), el kernel delega el informe del error a la capa de red mediante **ICMP**, devolviendo un mensaje de control Destination Unreachable (Port unreachable - Tipo 3, Código 3). hping3 computa la llegada de este mensaje ICMP como un paquete recibido, pero el contenido del mensaje certifica que el puerto se encuentra cerrado

Inciso 12

Investigue los distintos tipos de estado que puede tener una conexión TCP.

Ver:

https://users.cs.northwestern.edu/~agupta/cs340/project2/TCPIP_State_Transition_Diagram.pdf

Respuesta

Basado en el Diagrama de Transición de Estados TCP/IP (RFC 793), una conexión TCP opera como una máquina de estados finitos que evoluciona a través de 11 estados a lo largo de su ciclo de vida:

1. Estado Ficticio / Inicial

- **CLOSED:** Representa la ausencia total de conexión. Es un estado ficticio porque no existe ningún Bloque de Control de Transmisión (TCB) ni recursos asignados.

2. Fase de Establecimiento de Conexión (Apertura Pasiva y Activa)

- **LISTEN:** El servidor espera pacientemente una solicitud de conexión (SYN) proveniente de cualquier cliente y puerto remotos (Apertura pasiva / *Passive open*).
- **SYN-SENT:** El cliente ha iniciado la conexión enviando un segmento SYN y espera el correspondiente SYN-ACK de confirmación del servidor (Apertura activa / *Active open*).
- **SYN-RECEIVED (SYN_RCVD):** El servidor ha recibido el SYN del cliente, ha respondido con su propio SYN-ACK y aguarda la recepción del ACK final para confirmar la apertura.

3. Fase de Trasferencia de Datos

- **ESTABLISHED:** La conexión está completamente abierta e inicializada. Es el estado operativo normal donde se transmiten y entregan los datos de la capa de aplicación en ambas direcciones.

4. Fase de Cierre Activo (*Active Close*)

- Cuando la aplicación local decide iniciar la finalización de la conexión mediante un comando de cierre (`close()`):
 - **FIN-WAIT-1:** El host ha enviado un segmento FIN de terminación y espera la confirmación (ACK) del host remoto o una solicitud de cierre simultáneo.
 - **FIN-WAIT-2:** El host ya recibió el ACK correspondiente a su FIN, pero aún espera que el extremo remoto envíe su propio segmento FIN.
 - **TIME-WAIT:** El host ha recibido el FIN remoto y le envió el ACK final. Permanece en este estado durante un periodo equivalente a 2MSL (*Maximum Segment Lifetime*, típicamente entre 30

segundos y 2 minutos) para garantizar que el ACK final sea recibido por el extremo remoto y evitar que segmentos antiguos en tránsito interfieran con conexiones futuras.

5. Fase de Cierre Pasivo (*Passive Close*)

- Cuando el host recibe una solicitud de cierre (FIN) iniciada por el extremo remoto:
 - **CLOSE-WAIT:** El host recibió el FIN remoto y envió un ACK. Permanece en este estado esperando que la aplicación local notifique el fin del flujo de datos y ejecute la llamada de cierre.
 - **LAST-ACK:** La aplicación local ejecutó la llamada de cierre, por lo que el host envió su segmento FIN y aguarda la recepción del último ACK del host remoto para regresar a CLOSED.

6. Caso Especial: Cierre Simultáneo

- **CLOSING:** Ocurre cuando ambas partes inician el cierre al mismo tiempo enviando segmentos FIN de manera casi simultánea. El host aguarda la confirmación (ACK) para su propio FIN habiendo recibido ya el FIN de la otra parte.

Inciso 13

Dada la siguiente salida del comando **ss**, responda:

```
Netid State Recv-Q Send-Q Local Address:Port Peer Address:Port Process
tcp LISTEN 0 128 *:22 *: users:((("sshd",pid=468,fd=29))
tcp LISTEN 0 128 *:80 *: users:((("apache2",pid=991,fd=95))
udp LISTEN 0 128 163.10.5.222:53 *: users:((("named",pid=452,fd=10))
tcp ESTAB 0 0 163.10.5.222:59736 64.233.163.120:443 users:((("x-www-browser",pid=1079,fd=51))
tcp CLOSE-WAI T 0 0 163.10.5.222:41654 200.115.89.30:443
users:((("x-www-browser",pid=1079,fd=50))
tcp ESTAB 0 0 163.10.5.222:59737 64.233.163.120:443 users:((("x-www-browser",pid=1079,fd=55))
tcp ESTAB 0 0 163.10.5.222:33583 200.115.89.15:443 users:((("x-www-browser",pid=1079,fd=53))
tcp ESTAB 0 0 163.10.5.222:45293 64.233.190.99:443 users:((("x-www-browser",pid=1079,fd=59))
tcp LISTEN 0 128 *:25 *: users:((("postfix",pid=627,fd=3))
tcp ESTAB 0 0 127.0.0.1:22 127.0.0.1:41220 users:((("sshd",pid=1418,fd=3),
("sshd",pid=1416,fd=3))
tcp ESTAB 0 0 163.10.5.222:52952 64.233.190.94:443 users:((("x-www-browser",pid=1079,fd=29))
tcp TIME-WAIT 0 0 163.10.5.222:36676 54.149.207.17:443 users:((("x-www-browser",pid=1079,fd=3))
tcp ESTAB 0 0 163.10.5.222:52960 64.233.190.94:443 users:((("x-www-browser",pid=1079,fd=67))
tcp ESTAB 0 0 163.10.5.222:50521 200.115.89.57:443 users:((("x-www-browser",pid=1079,fd=69))
tcp SYN-SENT 0 0 163.10.5.222:52132 43.232.2.2:9500 users:((("x-www-browser",pid=1079,fd=70))
tcp ESTAB 0 0 127.0.0.1:41220 127.0.0.1:22 users:((("ssh",pid=1415,fd=3))
udp LISTEN 0 128 127.0.0.1:53 *: users:((("named",pid=452,fd=9))
```

1. ¿Cuántas conexiones hay establecidas?
2. ¿Cuántos puertos hay abiertos a la espera de posibles nuevas conexiones?

3. El cliente y el servidor de las comunicaciones HTTPS (puerto 443), ¿residen en la misma máquina?
4. El cliente y el servidor de la comunicación SSH (puerto 22), ¿residen en la misma máquina?
5. Liste los nombres de todos los procesos asociados con cada comunicación. Indique para cada uno si se trata de un proceso cliente o uno servidor.
6. ¿Cuáles conexiones tuvieron el cierre iniciado por el host local y cuáles por el remoto?
7. ¿Cuántas conexiones están aún pendientes por establecerse?

Inciso 13.1

Sí, actualmete hay 9 conexiones establecidas. Son aquellas con el estado `ESTAB`.

Observación importante: Los sockets 5 y 9 corresponden a los dos extremos de **una misma conexión local por SSH** (`127.0.0.1:22` `127.0.0.1:41220`). Por lo tanto, hay **9 sockets establecidos** correspondientes a **8 conexiones de red reales en el equipo** (7 hacia servidores web externos y 1 sesión SSH loopback local).

Inciso 13.2

Los puertos abiertos a la espera de nuevas conexiones son 5, los cuales tienen el valor `LISTEN`.

Total de puertos lógicos distintos: 4 puertos (el puerto `22/tcp`, `80/tcp`, `25/tcp` y `53/udp`, este último escuchando tanto en la IP pública como en localhost por el servicio DNS `named`).

Inciso 13.3

No, porque:

- En todas las líneas con destino al puerto `443` (como las asociadas al proceso `x-www-browser`), la columna **Local Address** tiene la IP `163.10.5.222` con puertos efímeros altos (por ejemplo `59736`, `59737`, `33583`, etc.), actuando como **cliente**.
- La columna **Peer Address** muestra IPs remotas externas (por ejemplo, `64.233.163.120:443`, `200.115.89.15:443`, `64.233.190.99:443`), que

corresponden a servidores web en Internet.

- Al tratarse de direcciones IP distintas y externas, el cliente (navegador local) y los servidores remotos residen en máquinas diferentes.

Inciso 13.4

Sí, debido a que:

- Al observar las conexiones establecidas sobre el puerto 22, aparecen dos entradas complementarias:
 1. `tcp ESTAB ... 127.0.0.1:22 -> 127.0.0.1:41220` (proceso servidor `sshd`)
 2. `tcp ESTAB ... 127.0.0.1:41220 -> 127.0.0.1:22` (proceso cliente `ssh`)
- Ambas partes se comunican utilizando la dirección de bucle local (`127.0.0.1` / **localhost**), lo que evidencia que tanto el cliente SSH como el demonio servidor SSH (`sshd`) se están ejecutando y comunicando dentro del mismo sistema.

Inciso 13.5

Proceso	Rol	Justificación
x-www-browser	Cliente	Inicia conexiones salientes hacia servidores web externos (puerto 443) y otros puertos remotos utilizando puertos efímeros locales.
ssh	Cliente	Proceso cliente SSH que se conecta localmente (<code>127.0.0.1:41220</code>) al servicio SSH del propio equipo.
sshd	Servidor	Demonio del servidor SSH a la escucha en el puerto 22 (<code>LISTEN</code>) y atendiendo la sesión establecida.
apache2	Servidor	Servidor web HTTP a la escucha en el puerto estándar 80 (<code>LISTEN</code>) a la espera de peticiones.
postfix	Servidor	Servidor de correo (MTA) a la escucha en el puerto estándar 25 (<code>LISTEN</code>) para recibir mensajes SMTP.

Proceso	Rol	Justificación
named	Servidor	Demonio DNS (BIND) a la escucha en el puerto 53 (LISTEN) sobre UDP para responder consultas de nombres.

Inciso 13.6

Cierre iniciado por el host local:

- **Conexión:** tcp TIME-WAIT 0 0 163.10.5.222:36676 -> 54.149.207.17:443
- **Justificación:** TIME-WAIT es exclusivo del extremo que realiza un **cierre activo** (*Active Close*). El host local envió el primer segmento FIN , transitó por FIN-WAIT-1 y FIN-WAIT-2 , y al recibir el FIN del servidor remoto y confirmarlo con un ACK , quedó en TIME-WAIT .
Cierre iniciado por el host remoto (*Passive Close*)
- **Conexión:** tcp CLOSE-WAIT 0 0 163.10.5.222:41654 -> 200.115.89.30:443
- **Justificación:** El estado CLOSE-WAIT corresponde al extremo que recibe un **cierre pasivo** (*Passive Close*). Esto significa que el servidor remoto envió primero un segmento FIN , el host local lo reconoció (ACK) y ahora el socket local está en espera de que la aplicación del sistema (x-www-browser) invoque su propia función close() para enviar su respectivo FIN y finalizar la sesión.

Inciso 13.7

- tcp SYN-SENT 0 0 163.10.5.222:52132 43.232.2.2:9500
 - El estado SYN-SENT representa una apertura activa (*Active Open*) donde el cliente local ya transmitió un segmento inicial SYN y se encuentra a la espera del SYN-ACK de confirmación del servidor remoto para continuar el *three-way handshake*. Por lo tanto, aún **no está establecida** y sigue **pendiente**

Inciso 14

Dadas las salidas de los siguientes comandos ejecutados en el cliente y el servidor, responder:

```

servidor# ss -natu | grep 110
tcp    LISTEN  0  0            *:110      *:.*
tcp    SYN-RECV  0  0      157.0.0.1:110  157.0.11.1:52843

cliente# ss -natu | grep 110
tcp    SYN-SENT  0  1      157.0.11.1:52843  157.0.0.1:110

```

1. Qué segmentos llegaron y cuáles se están perdiendo en la red?
2. ¿A qué protocolo de capa de aplicación y de transporte se está intentando conectar el cliente?
3. ¿Qué flags tendría seteado el segmento perdido?

Inciso 14.1

- **Segmento que llegó:** El segmento **SYN** enviado por el cliente hacia el servidor.
 - *Justificación:* El servidor pasó del estado pasivo **LISTEN** al estado **SYN-RECV** para el socket **157.0.0.1:110 <-> 157.0.11.1:52843**. Esto demuestra que el servidor recibió de forma efectiva la solicitud inicial de apertura enviada por el cliente.
- **Segmento que se está perdiendo:** El segmento **SYN+ACK** enviado por el servidor hacia el cliente.
 - *Justificación:* Al recibir el primer SYN, el servidor responde automáticamente con un segmento SYN+ACK. Sin embargo, el cliente continúa en estado **SYN-SENT** (y con valor **1** en su cola **Send-Q**, lo que indica un segmento no confirmado pendiente de retransmisión). Si el SYN+ACK hubiese llegado al host cliente, este habría procesado la confirmación, respondido con el ACK final y su estado habría cambiado a **ESTABLISHED**. Al seguir en **SYN-SENT**, se deduce que el SYN+ACK nunca llegó a su destino.

Inciso 14.2

Protocolo de capa de transporte: TCP.

Protocolo de capa de aplicación: POP3, ya que se está conectando al puerto estándar **110** sobre TCP.

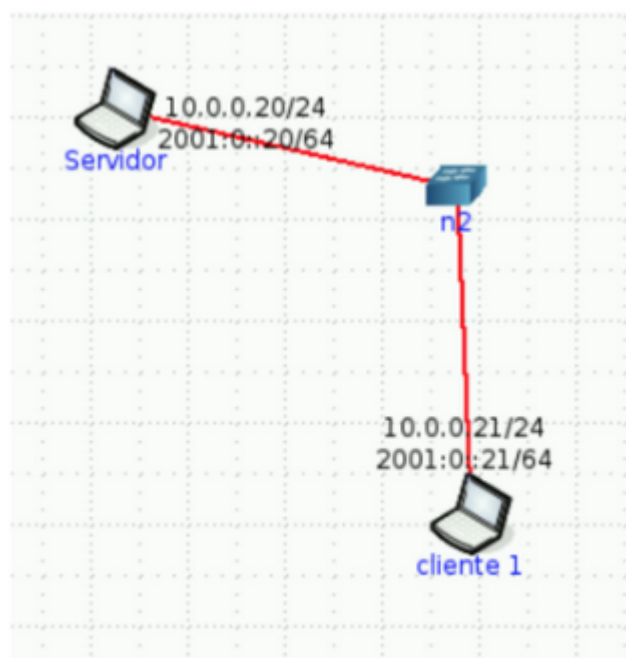
Inciso 14.3

- **SYN y ACK** (flags `SYN=1` y `ACK=1` , frecuentemente abreviado como **SYN+ACK** o **SA**).
- *Detalle:* El flag **SYN** se utiliza para sincronizar los números de secuencia iniciales del servidor, y el flag **ACK** confirma la recepción del segmento SYN previo emitido por el cliente.

Inciso 15

Use CORE para armar una topología como la siguiente, sobre la cual deberá realizar:

1. En ambos equipos inspeccionar el estado de las conexiones y mantener abiertas ambas ventanas con el comando corriendo para poder visualizar los cambios a medida que se realiza el ejercicio.
Ayuda: `watch -n1 'ss -nat'`.
2. En Servidor, utilice la herramienta `ncat` para levantar un servicio que escuche en el puerto 8001/TCP. Utilice la opción `-k` para que el servicio sea persistente. Verifique el estado de las conexiones

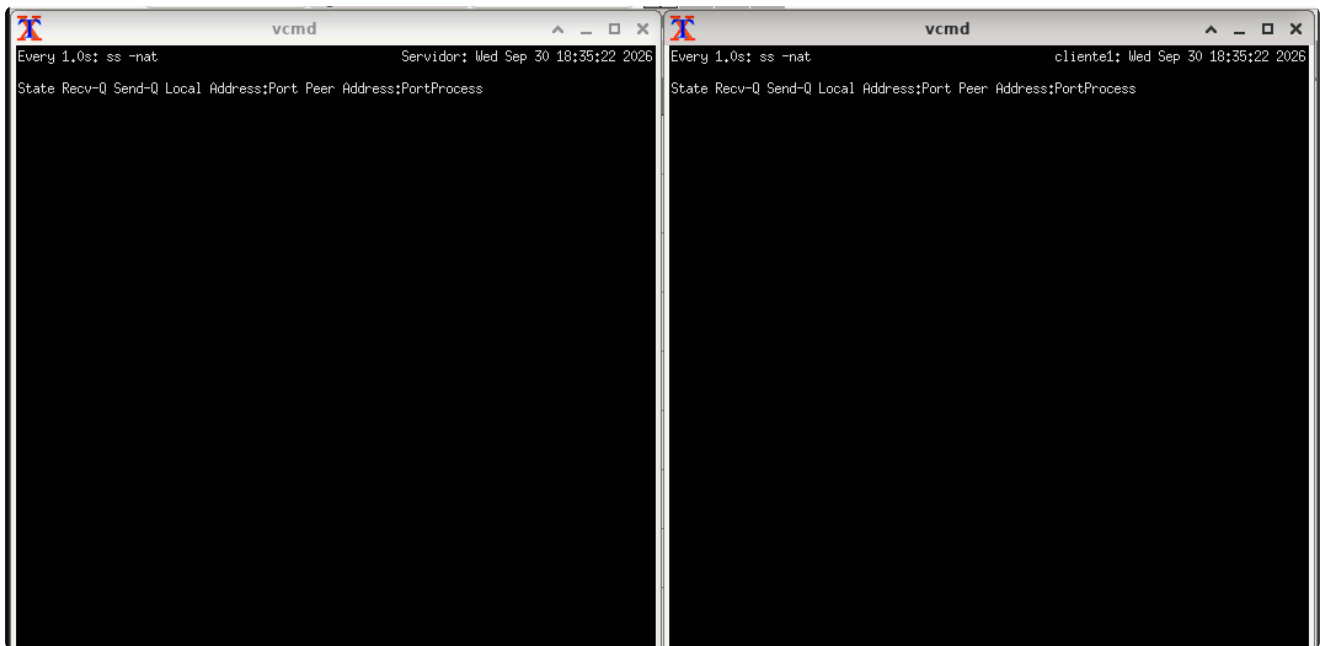


3. Desde CLIENTE1 conectarse a dicho servicio utilizando también la herramienta `ncat`. Inspeccione el estado de las conexiones.
4. Iniciar otra conexión desde CLIENTE1 de la misma manera que la anterior y verificar el estado de las conexiones. ¿De qué manera puede identificar cada conexión?
5. En base a lo observado en el item anterior, ¿es posible iniciar más de una conexión desde el cliente al servidor en el mismo puerto destino? ¿Por

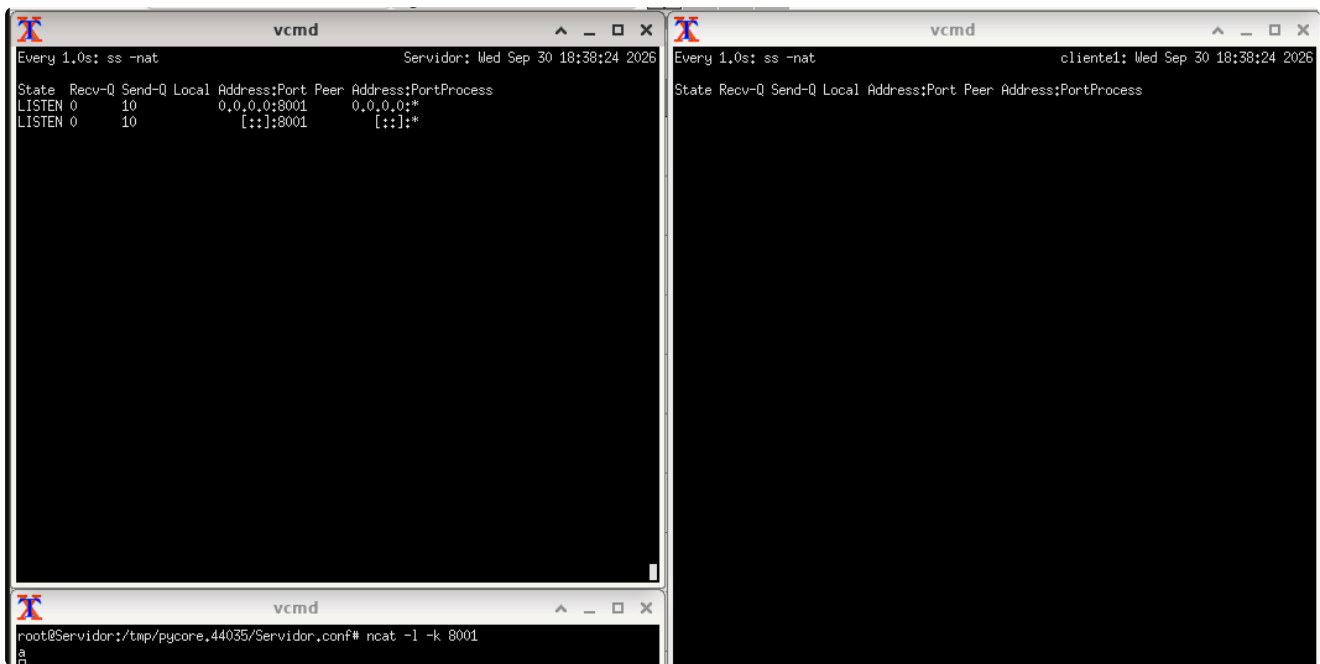
qué? ¿Cómo se garantiza que los datos de una conexión no se mezclarán con los de la otra?

6. Analice en el tráfico de red, los flags de los segmentos TCP que ocurren cuando:
 1. Cierra la última conexión establecida desde CLIENTE1. Evalúe los estados de las conexiones en ambos equipos.
 2. . Corta el servicio de ncat en el servidor (Ctrl+C). Evalúe los estados de las conexiones en ambos equipos.
 3. Cierra la conexión en el cliente. Evalúe nuevamente los estados de las conexiones

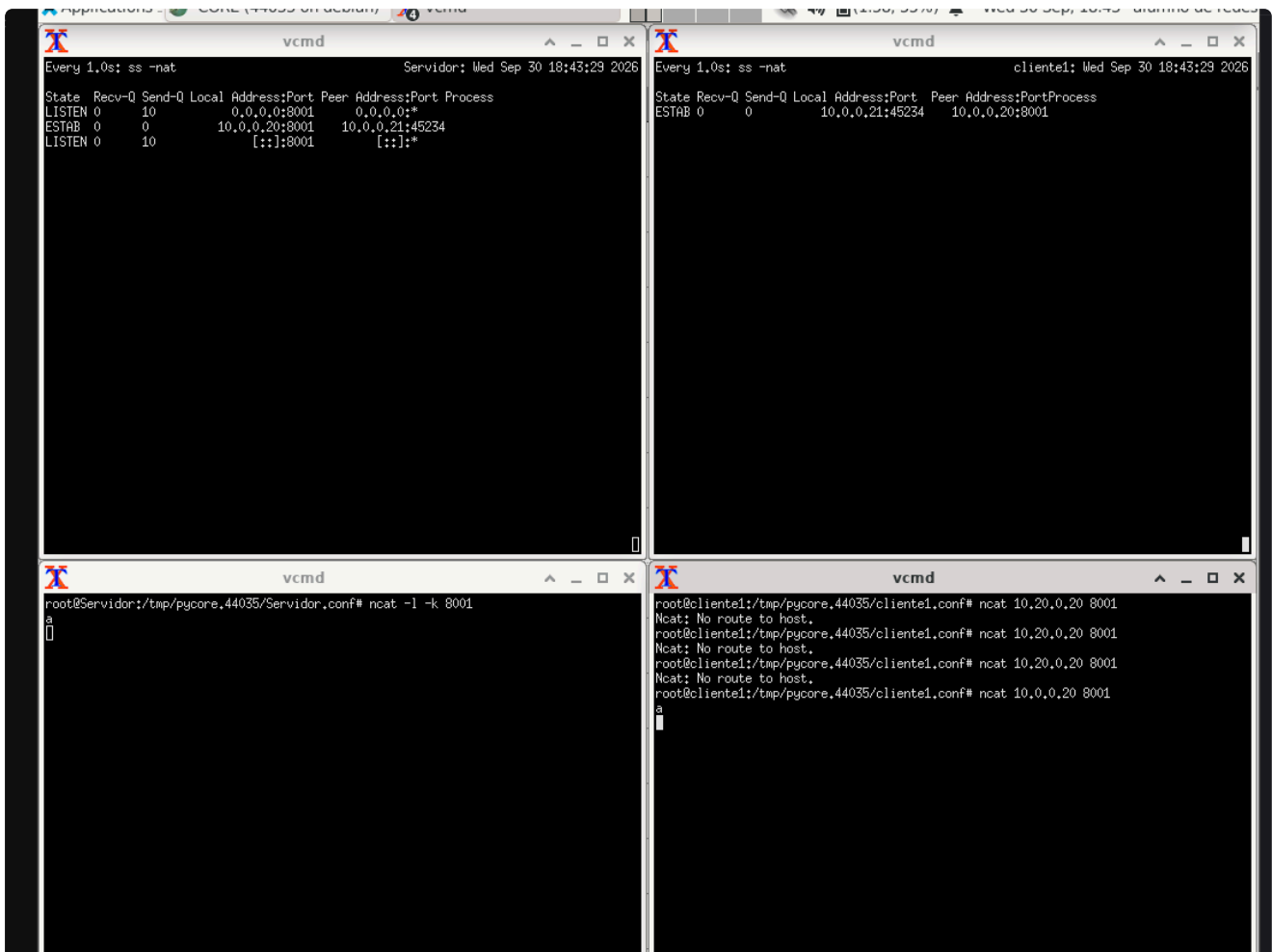
Inciso 15.1



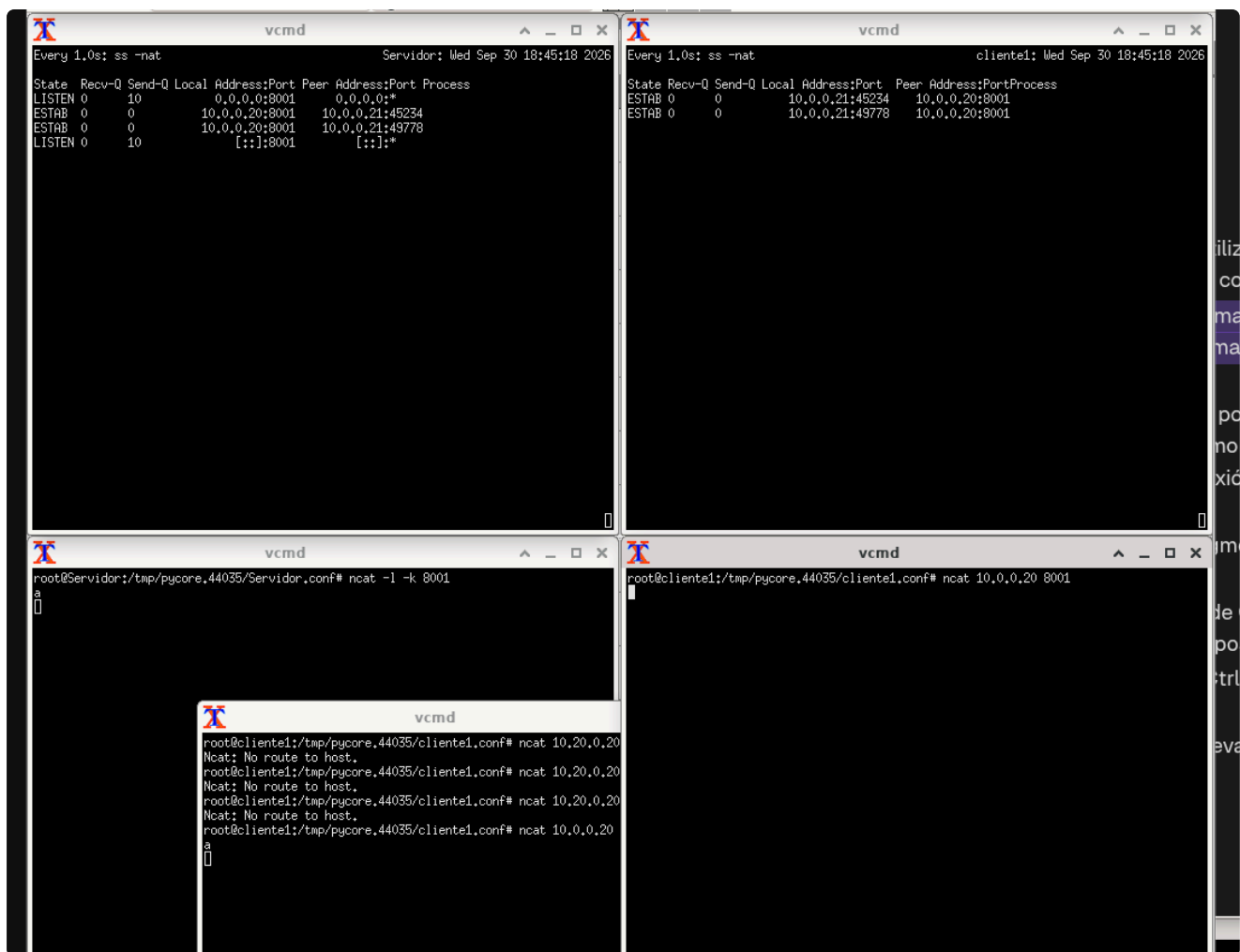
Inciso 15.2



Inciso 15.3



Inciso 15.4



Cada conexión se identifica de manera unívoca mediante su **4-upla** completa de transporte:

(IP Origen, Puerto Origen, IP Destino, Puerto Destino)

El elemento diferenciador: Dado que ambas conexiones comparten la misma IP de origen (10.0.0.21), la misma IP de destino (10.0.0.20) y el mismo puerto de destino (8001), el parámetro exacto que distingue a una conexión de la otra es el **número de puerto de origen (puerto efímero)** asignado por el sistema operativo del cliente a cada proceso.

Inciso 15.5

Sí, es posible hacerlo. Ya que la capa de transporte, una conexión TCP no está identificada únicamente por el puerto de destino del servidor. En su lugar, cada conexión TCP queda unívocamente determinada por la 4-upla de valores mencionada en el inciso anterior.

Cuando se inicia una segunda conexión desde `cliente1` (ejecutando otra instancia de `ncat / nc` en una nueva terminal), el sistema operativo del cliente le asigna de manera automática un **puerto de origen efímero**

diferente. Por lo tanto, aunque ambas conexiones compartan la misma IP de origen (10.0.0.21), la misma IP de destino (10.0.0.20) y el mismo puerto de destino (8001), las dos 4-uplas resultantes son distintas e independientes entre sí.